

SIMATS ENGINEERING

ASSESSMENT TOOL 1

COURSE CODE: CSA1402

COURSE NAME: COMPILER DESIGN

COURSE OUTCOME COVERED

CO3: *Analyze syntax-directed translation method using synthesized and inherited attributes to generate a Parser Tree. (BL4)*

Assessment Tool Weightage: 40%

QUESTIONS:5

Total Marks:100

Annexure A

EXPERIMENTS

Code of Conduct:

*I **R.Monika(192324281)** (Name / Reg No) certify that this submission is my original work and that I have adhered to the guidelines specified for this assessment.*

I understand that any violation of academic integrity rules will result in disciplinary action.

Signature of the student:

Monika.R

Annexure A

EXPERIMENTS

Q1. You are developing a compiler-based desk calculator that evaluates arithmetic expressions using syntax-directed definitions (SDD).

The system should:

- (i) Accept an arithmetic expression with operators +, -, *, / and parentheses
- (ii) Use syntax-directed evaluation to compute values based on operator precedence
- (iii) Optimize evaluation by reducing unnecessary temporary computations
- (iv) Display the final result of the expression

Demonstrate how SDD rules (synthesized attributes) are used for expression evaluation.

SDD

Grammar with semantic rules:

$E \rightarrow E + T \quad \{ E.val = E1.val + T.val \}$

$E \rightarrow E - T \quad \{ E.val = E1.val - T.val \}$

$E \rightarrow T \quad \{ E.val = T.val \}$

$T \rightarrow T * F \quad \{ T.val = T1.val * F.val \}$

$T \rightarrow T / F \quad \{ T.val = T1.val / F.val \}$

$T \rightarrow F \quad \{ T.val = F.val \}$

$F \rightarrow (E) \quad \{ F.val = E.val \}$

$F \rightarrow \text{digit} \quad \{ F.val = \text{digit} \}$

ANSWERS: Syntax-Directed Definition (SDD) – Arithmetic Expression Evaluation

Aim

To implement a compiler-based calculator using **syntax-directed definitions (SDD)** and evaluate arithmetic expressions with correct operator precedence.

C Program

```
#include <stdio.h>
#include <ctype.h>
char exp[100];
int pos = 0;
int E();
int T();
int F();
int E() {
```

```

int value = T();
while (exp[pos] == '+' || exp[pos] == '-') {
    char op = exp[pos++];
    int value2 = T();
    if (op == '+')
        value += value2;
    else
        value -= value2;
}
return value;
}
int T() {
    int value = F();
    while (exp[pos] == '*' || exp[pos] == '/') {
        char op = exp[pos++];
        int value2 = F();
        if (op == '*')
            value *= value2;
        else
            value /= value2;
    }
    return value;
}
int F() {
    int value;
    if (exp[pos] == '(') {
        pos++;
        value = E();
        pos++;
    } else {
        value = 0;
        while (isdigit(exp[pos]))
            value = value * 10 + (exp[pos++] - '0');
    }
    return value;
}

```

```

int main() {
    printf("Enter arithmetic expression: ");
    scanf("%s", exp);
    printf("Result = %d\n", E());
    return 0;
}

```

Sample Input and Output

Enter arithmetic expression: 2+3*4

Result = 14

OUTPUT:

The screenshot displays an IDE with a C++ source file named `co3_at1_q1.cpp` and its execution output in a separate window.

Source Code:

```

1 #include <stdio.h>
2 #include <ctype.h>
3 char exp[100];
4 int pos = 0;
5 int E();
6 int T();
7 int F();
8 int E() {
9     int value = T();
10    while (exp[pos] == '+' || exp[pos] == '-') {
11        char op = exp[pos++];
12        int value2 = T();
13        if (op == '+')
14            value += value2;
15        else
16            value -= value2;
17    }
18    return value;
19 }
20 int T() {
21     int value = F();
22     while (exp[pos] == '*' || exp[pos] == '/') {
23         char op = exp[pos++];
24         int value2 = F();
25         if (op == '*')
26             value *= value2;
27         else
28             value /= value2;
29     }
30    return value;
31 }
32 int F() {
33     int value;
34     if (exp[pos] == '(') {
35         pos++;
36         value = E();
37         pos++;
38     } else {
39         value = 0;
40         while (isdigit(exp[pos]))
41             value = value * 10 + (exp[pos++] - '0');
42     }
43    return value;
44 }
45
46 int main() {
47     printf("Enter arithmetic expression: ");
48     scanf("%s", exp);
49     printf("Result = %d\n", E());
50     return 0;
51 }

```

Execution Output:

```

Enter arithmetic expression: (2+3)*4
Result = 20

-----
Process exited after 19.99 seconds with return value 0
Press any key to continue . . .

```

The IDE interface includes a menu bar (File, Edit, Search, View, Project, Execute, Tools, AStyle, Window, Help), a toolbar, a project explorer showing `co3_at1_q1.cpp`, a compiler status bar (Compiler (1), Resources, Compile Log, Debug, Find Results, Console, Close), and a status bar at the bottom showing line and column information (Line: 1 Col: 19 Sel: 0 Lines: 51 Length: 1033 Insert) and system information (Done parsing in 0.046 seconds, 20:58, 07-08-2026).

Result

Thus, the arithmetic expression was successfully evaluated using **syntax-directed evaluation**, and operator precedence was correctly maintained.

Q2. In a compiler, postfix expressions are evaluated during code generation using stack-based bottom-up evaluation.

Write a C program that:

- (i) Evaluates a postfix expression
- (ii) Demonstrates S-attributed definition evaluation
- (iii) Shows how intermediate results are computed

ANSWER: Postfix Expression Evaluation – S-Attributed Definition

Aim

To evaluate a postfix expression using a stack-based bottom-up evaluation and demonstrate an S-attributed definition.

C Program

```
#include <stdio.h>
#include <ctype.h>
int stack[100];
int top = -1;
void push(int x) {
    stack[++top] = x;
}
int pop() {
    return stack[top--];
}
int main() {
    char exp[100];
    int i, a, b, result;
    printf("Enter postfix expression: ");
    scanf("%s", exp);
    for (i = 0; exp[i] != '\0'; i++) {
        if (isdigit(exp[i])) {
            push(exp[i] - '0');
        } else {
            b = pop();
            a = pop();
            switch (exp[i]) {
                case '+': result = a + b; break;
                case '-': result = a - b; break;
```

```

        case '*': result = a * b; break;
        case '/': result = a / b; break;
    }
    push(result);
}
}
printf("Result = %d\n", pop());
return 0;
}

```

Sample Input and Output

Enter postfix expression: 234*+ ;

Result = 14

OUTPUT:

The screenshot displays an IDE with a C++ program for evaluating postfix expressions. The code uses a stack to store operands and a switch statement to handle the operators '+', '-', '*', and '/'. The main function prompts the user to enter a postfix expression, which is then processed character by character. The output window shows the execution results for the input '234*+'.

```

1  #include <stdio.h>
2  #include <ctype.h>
3  int stack[100];
4  int top = -1;
5  void push(int x) {
6      stack[++top] = x;
7  }
8  int pop() {
9      return stack[top--];
10 }
11 int main() {
12     char exp[100];
13     int i, a, b, result;
14     printf("Enter postfix expression: ");
15     scanf("%s", exp);
16     for (i = 0; exp[i] != '\0'; i++) {
17         if (isdigit(exp[i])) {
18             push(exp[i] - '0');
19         } else {
20             b = pop();
21             a = pop();
22             switch (exp[i]) {
23                 case '+': result = a + b; break;
24                 case '-': result = a - b; break;
25                 case '*': result = a * b; break;
26                 case '/': result = a / b; break;
27             }
28             push(result);
29         }
30     }
31     printf("Result = %d\n", pop());
32     return 0;
33 }

```

Execution Output:

```

Enter postfix expression: 234*+
Result = 14

-----
Process exited after 31.71 seconds with return value
0
Press any key to continue . . .

```

Result

Thus, the postfix expression was successfully evaluated using stack-based bottom-up evaluation and synthesized attributes.

Q3. You are designing a parser that uses L-attributed definitions, where inherited attributes are passed from parent to child.

Write a C program that:

- (i). Evaluates a simple expression like $a + b * c$
- (ii) Uses function parameters to simulate inherited attributes
- (iii). Ensures correct precedence handling
- (iv). Demonstrates top-down evaluation logic

Explain how inherited attributes influence computation.

ANSWER: L-Attributed Definition – Inherited Attributes

Aim

To implement an expression evaluator using L-attributed definitions, function parameters, inherited attributes, correct precedence, and top-down evaluation.

C Program

```
#include <stdio.h>
#include <ctype.h>
char exp[100];
int pos = 0;
int E();
int T(int inherited);
int F();
int E() {
    int value = T(0);
    return value;
}
int T(int inherited) {
    int value = F();
    while (exp[pos] == '*') {
        pos++;
        value = value * F();
    }
    return inherited + value;
}
int F() {
    int value = 0;
    if (exp[pos] == '(') {
```

```

    pos++;
    value = E();
    pos++;
} else {
    while (isdigit(exp[pos]))
        value = value * 10 + (exp[pos++] - '0');
}
return value;
}

int main() {
    printf("Enter expression: ");
    scanf("%s", exp);
    printf("Result = %d\n", E());
    return 0;
}

```

Sample Output

Enter expression: 2+3*4

Result = 14

OUTPUT:

The screenshot displays a C++ IDE with the source code for a calculator program. The code defines a function `E()` that evaluates expressions using L-attributed evaluation. The `main` function prompts the user for an expression and prints the result. The output window shows the program's execution for the input "2+5-8*3", resulting in "Result = 2".

```

1  #include <stdio.h>
2  #include <ctype.h>
3  char exp[100];
4  int pos = 0;
5  int E();
6  int T(int inherited);
7  int F();
8  int E() {
9      int value = T(0);
10     return value;
11 }
12 int T(int inherited) {
13     int value = F();
14     while (exp[pos] == '*' || exp[pos] == '/') {
15         pos++;
16         value = value * F();
17     }
18     return inherited + value;
19 }
20 int F() {
21     int value = 0;
22     if (exp[pos] == '(') {
23         pos++;
24         value = E();
25         pos++;
26     } else {
27         while (isdigit(exp[pos]))
28             value = value * 10 + (exp[pos++] - '0');
29     }
30     return value;
31 }
32 int main() {
33     printf("Enter expression: ");
34     scanf("%s", exp);
35     printf("Result = %d\n", E());
36     return 0;
37 }

```

Enter expression: 2+5-8*3
Result = 2

Process exited after 10.56 seconds with return value 0
Press any key to continue . . .

Result

The program successfully demonstrates L-attributed evaluation using function parameters as inherited attributes.

Q4.In a compiler, it is important to check whether two type expressions are equivalent.

Write a C program that:

(i).Accepts two type expressions (e.g., int, float, int*)

(ii).Compares them for **type equivalence**

(iii).Displays whether they are:

(a).Equivalent

(b).Not equivalent

Extend the logic to handle pointer types (int*, float*).

ANSWER: Type Equivalence

Aim

To compare two type expressions and determine whether they are equivalent, including pointer types.

C Program

```
#include <stdio.h>
#include <string.h>
int typeEquivalent(char t1[], char t2[]) {
    if (strcmp(t1, t2) == 0)
        return 1;
    if ((strcmp(t1, "int*") == 0 && strcmp(t2, "int*") == 0) ||
        (strcmp(t1, "float*") == 0 && strcmp(t2, "float*") == 0))
        return 1;
    return 0;
}
int main() {
    char type1[20], type2[20];
    printf("Enter first type: ");
    scanf("%s", type1);
    printf("Enter second type: ");
    scanf("%s", type2);
    if (typeEquivalent(type1, type2))
        printf("Equivalent\n");
    else
        printf("Not Equivalent\n");
    return 0;
}
```

Sample Output:

Enter first type: int

Enter second type: int

Equivalent

OUTPUT:

The screenshot shows an IDE with a C++ project. The code defines a function `typeEquivalent` that compares two type strings. It uses `strcmp` to check for exact matches and `strcmp` with `"int*"` and `"float*"` to check for pointer types. The `main` function prompts the user to enter two types and prints the result. The terminal window shows the program's execution: it prompts for the first type (int), the second type (float), and outputs "Not Equivalent". It also shows the process exiting after 46.59 seconds with a return value of 0.

```
1 #include <stdio.h>
2 #include <string.h>
3 int typeEquivalent(char t1[], char t2[]) {
4     if (strcmp(t1, t2) == 0)
5         return 1;
6     if ((strcmp(t1, "int*") == 0 && strcmp(t2, "int*") == 0) ||
7         (strcmp(t1, "float*") == 0 && strcmp(t2, "float*") == 0))
8         return 1;
9     return 0;
10 }
11 int main() {
12     char type1[20], type2[20];
13     printf("Enter first type: ");
14     scanf("%s", type1);
15     printf("Enter second type: ");
16     scanf("%s", type2);
17     if (typeEquivalent(type1, type2))
18         printf("Equivalent\n");
19     else
20         printf("Not Equivalent\n");
21     return 0;
22 }
```

```
Enter first type: int*
Enter second type: float*
Not Equivalent

-----
Process exited after 46.59 seconds with return value 0
Press any key to continue . . .
```

Explanation

The program compares the structure of the type expressions.

Examples:

int ≡ int → Equivalent

float ≡ float → Equivalent

int ≠ float → Not Equivalent

int* ≡ int* → Equivalent

int* ≠ float* → Not Equivalent

Result

Thus, the program successfully determines type equivalence for basic and pointer types.

Q5. You are building a semantic analyzer that detects type errors in arithmetic expressions. Write a C program that:

- (i). Accepts two operands and their types
- (ii). Checks whether the operation (e.g., +, *) is valid
- (iii). Reports:
 - (a). Valid expression
 - (b). Type error (e.g., `int + char*`)

Demonstrate how compilers detect semantic errors during type checking

ANSWER: Semantic Analyzer – Type Error Detection

Aim

To develop a semantic analyzer that detects **type errors in arithmetic expressions**.

The question specifically requires:

- Accept two operands and their types
- Check whether an operation such as + or * is valid
- Report a valid expression or type error

C Program

```
#include <stdio.h>
#include <string.h>
int validOperation(char type1[], char op, char type2[]) {
    if (strcmp(type1, "int") == 0 && strcmp(type2, "int") == 0)
        return 1;
    if (strcmp(type1, "float") == 0 && strcmp(type2, "float") == 0)
        return 1;
    return 0;
}
int main() {
    char type1[20], type2[20], op;
    printf("Enter first operand type: ");
    scanf("%s", type1);
    printf("Enter operator (+, *, -, /): ");
    scanf(" %c", &op);
    printf("Enter second operand type: ");
    scanf("%s", type2);
    if (validOperation(type1, op, type2))
        printf("Valid expression\n");
}
```

```

else
    printf("Type Error: Incompatible operands\n");
return 0;
}

```

Sample Input and Output

Enter first operand type: int

Enter operator (+, *, -, /): +

Enter second operand type: int

Valid expression

OUTPUT:

The screenshot shows an IDE window with the following C++ code:

```

1 #include <stdio.h>
2 #include <string.h>
3 int validOperation(char type1[], char op, char type2[]) {
4     if (strcmp(type1, "int") == 0 && strcmp(type2, "int") == 0)
5         return 1;
6     if (strcmp(type1, "float") == 0 && strcmp(type2, "float") == 0)
7         return 1;
8     return 0;
9 }
10 int main() {
11     char type1[20], type2[20], op;
12     printf("Enter first operand type: ");
13     scanf("%s", type1);
14     printf("Enter operator (+, *, -, /): ");
15     scanf("%c", &op);
16     printf("Enter second operand type: ");
17     scanf("%s", type2);
18     if (validOperation(type1, op, type2))
19         printf("Valid expression\n");
20     else
21         printf("Type Error: Incompatible operands\n");
22     return 0;
23 }

```

The output window shows the following text:

```

Enter first operand type: int
Enter operator (+, *, -, /): +
Enter second operand type: int
Valid expression

-----
Process exited after 26.01 seconds with return value 0
Press any key to continue . . .

```

Result

Thus, the semantic analyzer successfully detects **type errors during semantic/type checking**.

RUBRICS FOR CALCULATION

Criteria	Weightage	Level 4 – Exemplary	Level 3 – Proficient	Level 2 – Developing	Level 1 – Beginning
Understanding of Concepts	25	Demonstrates thorough understanding and effectively integrates multiple concepts/technologies	Good understanding with proper integration of most concepts	Basic understanding; limited or partial integration	Poor understanding; unable to integrate concepts
Experimental Design	30	Well-planned, systematic implementation with optimal solution	Correct implementation with minor issues	Basic implementation with errors or inefficiencies	Incorrect or incomplete implementation
Use of Tools	20	Effective and advanced use of tools/technologies with proper configuration	Appropriate use of tools with correct execution	Limited or inconsistent use of tools	Incorrect or minimal use of tools
Analysis of Results	15	Insightful analysis with accurate interpretation and validation of results	Good analysis with reasonable interpretation	Basic analysis with limited insights	Poor or incorrect analysis of results
Documentation	10	Clear, well-structured documentation with diagrams, code, and results	Organized documentation with necessary details	Basic documentation with missing details	Poor or incomplete documentation